Experiment 7: Automating Build and Testing Processes Using GitHub Actions Workflows

Collaborative Development and DevOps (1BCS303) - Laboratory Manual

1. Aim

To automate the build and testing process of a **Python** application, written in **Visual Studio Code**, using GitHub Actions workflows.

2. Learning Objectives

- Set up Visual Studio Code for Python and create project files and folders in it.
- Write a simple Python function and automated tests using pytest.
- Add a build step and a test step to a CI workflow.
- Run automated tests as part of the pipeline.
- Understand and configure a matrix build across multiple Python versions.
- Add a status badge to the repository README.

3. Prerequisites

- Completion of Experiment 6 (a GitHub repository, e.g., ci-lab-6, already cloned on your computer).
- Git installed and configured (Experiment 1).
- Python 3.10 or later installed (check with `python --version`).

4. Requirements / Software

- Visual Studio Code (free) with the Python extension by Microsoft
- Python 3 and pip
- Git and a GitHub account
- pytest (Python test framework)

5. Important Concepts Before Starting

5.1 Build vs Test Stage

The build stage prepares the application (e.g., installing dependencies, checking that the code compiles). The test stage runs automated tests to verify the application behaves correctly. Python is interpreted, so the "build" here means installing dependencies and checking the code for syntax errors.

5.2 Matrix Build

A matrix build runs the same job multiple times with different configurations, such as multiple Python versions, to verify compatibility.

5.3 What is pytest?

pytest is a Python testing framework. It automatically finds files named `test_*.py` and runs every function whose name starts with `test_`. A test passes if its `assert` statements are true.

6. Experiment Procedure

Part A - Set Up Visual Studio Code

Step 1: Install Visual Studio Code

- Download VS Code from https://code.visualstudio.com and run the installer with default options.
- On Windows, tick **Add to PATH** and **Add "Open with Code"** if shown.

Step 2: Install the Python extension

- Click the **Extensions** icon in the left sidebar (or press Ctrl+Shift+X).
- Search for **Python** and install the one published by **Microsoft**.

Step 3: Open your repository folder in VS Code

- Go to **File -> Open Folder...** and select your cloned repository folder (e.g., ci-lab-6).
- Click **Yes, I trust the authors** if asked.
- Or, from Git Bash inside the repository folder, run: `code .`

Step 4: Open the integrated terminal

Press **Ctrl +** ` (the backtick key below Esc) or use **Terminal -> New Terminal**. A terminal opens at the bottom, already inside your project folder. Check the Python version:

```
python --version
```

On Linux/macOS use `python3 --version`. If Python is not found on Windows, try `py --version`.

Part B - Create Files in VS Code

The Explorer panel on the left (Ctrl+Shift+E) shows the files in your project. There are several ways to create a file in VS Code:

Method	How to do it
Explorer button (easiest)	Hover over the Explorer panel, click the **New File** icon (page with a + sign), type the file name with its extension (e.g., calculator.py) and press Enter.
Right-click menu	Right-click an empty area in the Explorer panel -> **New File** -> type the name -> Enter.
Menu / keyboard	Press **Ctrl + N** (File -> New Text File), type the code, press **Ctrl + S**, choose the project folder and enter the file name with the extension.
Terminal	Type `echo. > calculator.py` (Windows CMD) or `touch calculator.py` (Git Bash / Linux / macOS).

To create a folder: click the **New Folder** icon in the Explorer panel and type the folder name. To create a file inside a folder, right-click that folder and choose **New File**. Always save with **Ctrl + S**; an unsaved file shows a white dot on its tab.

Step 5: Create calculator.py

Create a new file named `calculator.py` in the repository root and type:

```
def add(a, b):
    return a + b


def subtract(a, b):
    return a - b


if __name__ == "__main__":
    print("2 + 3 =", add(2, 3))
```

Step 6: Create test_calculator.py

Create a second file named `test_calculator.py` (the name must start with `test_`):

```
from calculator import add, subtract


def test_add():
    assert add(1, 2) == 3


def test_subtract():
    assert subtract(5, 3) == 2
```

Step 7: Create requirements.txt

Create a file named `requirements.txt` containing one line:

```
pytest
```

Step 8: Create .gitignore (recommended)

Create `.gitignore` so temporary files are not committed:

```
__pycache__/
.pytest_cache/
.venv/
```

Part C - Run Tests Locally First

Step 9: Install pytest

In the VS Code terminal run:

```
pip install -r requirements.txt
```

Step 10: Run the program and the tests

```
python calculator.py
pytest
```

Sample Output:

```
2 + 3 = 5

======================= test session starts =======================
collected 2 items

test_calculator.py ..                                      [100%]

======================== 2 passed in 0.02s ========================
```

Optional: VS Code can also run tests visually. Press Ctrl+Shift+P, choose **Python: Configure Tests**, select **pytest**, and use the **Testing** (flask icon) panel in the sidebar.

Part D - Create the Workflow File in VS Code

Step 11: Create the workflow folders and file

- In the Explorer panel, click **New Folder** and type **.github**.
- Right-click **.github** -> **New Folder** -> type **workflows**.

- Right-click **workflows** -> **New File** -> type **ci.yml**.
- Or type **.github/workflows/ci.yml** as the file name in one step; VS Code creates the folders automatically.

If `ci.yml` already exists from Experiment 6, open it and replace its entire content with the following:

```
name: Build and Test

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.10", "3.12"]

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Build (check syntax)
        run: python -m py_compile calculator.py

      - name: Run tests
        run: pytest
```

Note: write the versions in quotes ("3.10"). Without quotes YAML reads 3.10 as the number 3.1, which is wrong. Use spaces for indentation, never tabs (VS Code shows the indentation setting at the bottom-right: choose **Spaces: 2**).

Step 12: Commit and push

Using the terminal:

```
git add .
git commit -m "Add Python build and test workflow with matrix strategy"
git push origin main
```

Or using VS Code: click the **Source Control** icon (Ctrl+Shift+G), click **+** next to the changed files to stage them, type a commit message, click **Commit**, then **Sync Changes** / **Push**.

Part E - Observe, Break, Fix

Step 13: Observe the Actions run

Open the repository on GitHub and click the **Actions** tab. Confirm that two parallel jobs run, one for each Python version in the matrix (3.10 and 3.12). Open a job and expand the **Run tests** step to see

pytest output.

Step 14: Introduce a failing test

In VS Code, open `test_calculator.py` and change the expected value of test_add from 3 to 4:

```
def test_add():
    assert add(1, 2) == 4
```

```
git add test_calculator.py
git commit -m "Introduce failing test"
git push origin main
```

The run now fails with a red X. The log shows the assertion error:

```
    def test_add():
>       assert add(1, 2) == 4
E       assert 3 == 4
E        +  where 3 = add(1, 2)

FAILED test_calculator.py::test_add - assert 3 == 4
=================== 1 failed, 1 passed ===================
```

Step 15: Fix the test

Change the expected value back to 3, save, commit and push. Confirm the workflow shows a green check mark again.

Step 16: Add a status badge

- On the Actions tab, open the workflow, click the three-dots (...) menu and select **Create status badge**.
- Copy the Markdown snippet.
- In VS Code, create or open **README.md** and paste the snippet at the top.

```
![Build and Test](https://github.com/USER/REPO/actions/workflows/ci.yml/badge.svg)
```

Commit and push the README update. The badge shows **passing** in green when the latest run succeeds.

7. Common Errors and Solutions

Problem	Cause and Solution
ModuleNotFoundError: No module named pytest	pytest is not installed. Run `pip install -r requirements.txt` locally, and make sure requirements.txt exists and is committed so the workflow can install it.
No tests ran / collected 0 items	The test file name must start with `test_` and test functions must start with `test_`. Save the file with .py extension.
ModuleNotFoundError: No module named calculator	The test file and calculator.py must be in the same folder. Check spelling of the file name.
VS Code says "python is not recognized"	Python is not on PATH. Reinstall Python and tick **Add Python to PATH**, then restart VS Code. Try `py` or `python3`.
Tests pass locally but fail in Actions	A file may not have been committed (check `git status`) or requirements.txt is missing a package.

Problem	Cause and Solution
Python 3.10 is read as 3.1 in the matrix	Put version numbers in quotes: ["3.10", "3.12"].
Matrix job fails for only one version	Some code may rely on a feature only present in newer versions; check the failing version log for specifics.
YAML indentation error	Use spaces, not tabs, and match the indentation in the example exactly.

8. Important Terms

Term	Meaning
Matrix build	Running the same job with multiple parameter combinations.
pytest	A Python framework that finds and runs test functions automatically.
assert	A Python statement that fails the test if the condition is false.
requirements.txt	A text file listing the Python packages a project needs.
Artifact	A file produced by a workflow run that can be downloaded or passed between jobs.
Badge	An image showing the current status of a workflow, embeddable in a README.
CI/CD	Continuous Integration / Continuous Delivery or Deployment.

9. Expected Result

- Python files created in Visual Studio Code and tested locally with pytest.
- Automated build and test steps added to the CI workflow.
- Tests executed successfully across a matrix of Python versions (3.10 and 3.12).
- A failing test correctly detected and then fixed.
- A build status badge added to the README.

10. Student Observation / Screenshot Checklist

- VS Code with the Python extension installed
- Explorer panel showing calculator.py, test_calculator.py, requirements.txt
- Local pytest output showing 2 passed
- ci.yml open in VS Code
- Actions tab showing both matrix jobs passing
- Failed run (red X) with the assertion error
- Fixed run (green check mark)
- README with the status badge

11. Short Revision - Remember These Commands

```
pip install -r requirements.txt   -> Install pytest and other packages
python calculator.py              -> Run the Python program
pytest                            -> Run tests locally
git push origin main              -> Trigger the build-and-test workflow

VS Code: Ctrl+`  terminal | Ctrl+S  save | Ctrl+Shift+E  Explorer
         Ctrl+Shift+G  Source Control | Ctrl+Shift+X  Extensions
```

12. Final Takeaway

Automated build and test steps in a CI workflow catch problems immediately after every push. A matrix strategy extends this to multiple Python versions in parallel, and a status badge communicates the current health of the project at a glance. Visual Studio Code makes it easy to create, edit, test and commit all project files in one place.

13. Result

Thus, the build and testing process of a Python application created in Visual Studio Code was successfully automated using GitHub Actions workflows.